

Nombre: Miyasaki Sato Yuichi Vicente

Materia: Sistemas Distribuidos

Paso 1: La historia

Hora: 8:46 p.m

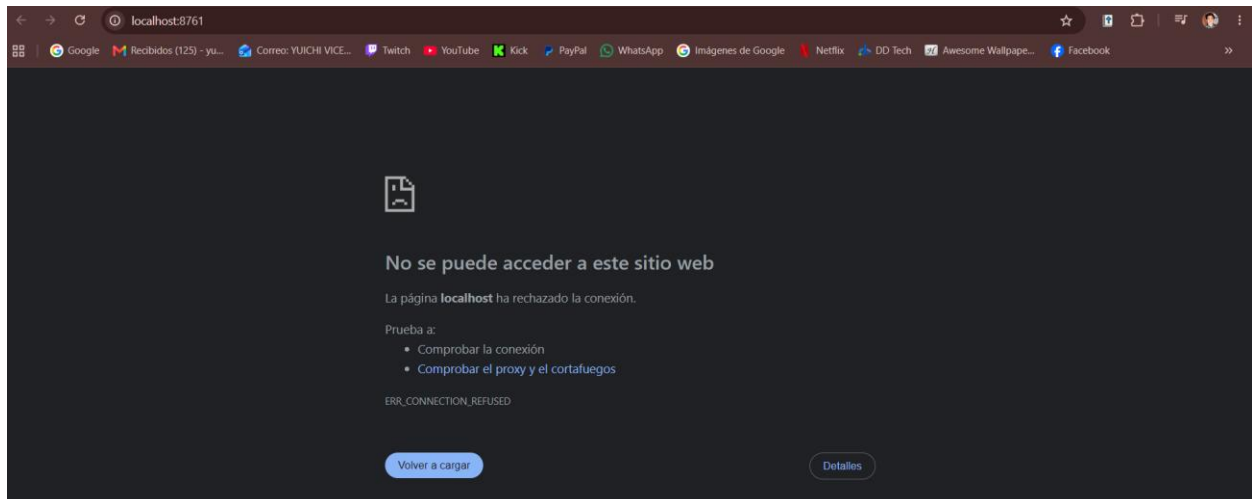
Al pasar de llamada a método a HTTP se perdió el todo o nada. En el monolito, el cargo y el abono estaban dentro de un mismo `@Transactional` y usaban la misma base de datos: si el abono fallaba, la base deshacía también el cargo.

Al partir el banco en cinco servicios, transferencia ya no llama a un método. Ahora hace una petición HTTP a cuenta, que es otro proceso con su propia base. Cada servicio confirma solo lo suyo, así que si el cargo ya quedó confirmado en cuenta y después falla el abono, el cargo no se deshace y el dinero queda descuadrado.

También noté que las direcciones de los servicios estaban escritas a mano en el `docker-compose.yml`, por ejemplo `CUENTA_URL: http://cuenta:8080`.

Paso 2: Directorio

Hora: 8:55 p.m



1. En `DiscoveryApplication.java` agregué el import y la anotación `@EnableEurekaServer` para encender el servidor Eureka.
2. En el `application.yml` de discovery agregué `register-with-eureka: false` y `fetch-registry: false`, para que el directorio no se anote a sí mismo.
3. En los cinco `pom.xml` descomenté la dependencia `spring-cloud-starter-netflix-eureka-client`, que es la que anota a cada servicio y manda los heartbeats.
4. En `docker-compose.yml` descomenté el servicio `discovery` y las líneas de `depends_on`, para que los servicios esperen a que el directorio esté sano.

- Reconstruí con docker compose up -d --build y confirmé con docker compose ps que los siete contenedores quedaran en healthy.

Volví a abrir <http://localhost:8761> y ya cargó el tablero de Eureka. Aparecieron cinco servicios en UP, cada uno con una copia: CUENTA, MOVIMIENTO, NOTIFICACION, SPEI y TRANSFERENCIA. También salió el letrero rojo de que el modo de autopreservación está apagado, lo cual es normal porque lo apagamos a propósito para que Eureka borre rápido a los servicios caídos.

The screenshot shows the Spring Eureka dashboard in a web browser. The top navigation bar includes the Eureka logo and a 'HOME' link. The 'System Status' section displays the following information:

Environment	test	Current time	2026-10-05T03:04:03 +0000
Data center	default	Uptime	00:00
		Lease expiration enabled	true
		Renews threshold	10
		Renews (last min)	0

Below the status section, a red warning message states: "THE SELF PRESERVATION MODE IS TURNED OFF. THIS MAY NOT PROTECT INSTANCE EXPIRY IN CASE OF NETWORK/OTHER PROBLEMS." The 'DS Replicas' section shows 'localhost'. The 'Instances currently registered with Eureka' section contains a table with the following data:

Application	AMIs	Availability Zones	Status
CUENTA	n/a (1)	(1)	UP (1) - 856d1d6dd9ec:cuenta:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 54c552bcc100:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 7ebde1652e55:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 011bc8930ad8:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 9918b2097345:transferencia:8080

XML de CUENTA. En <http://localhost:8761/eureka/apps/CUENTA> revisé lo que el directorio sabe de cuenta:

The screenshot shows the XML response from the Eureka application endpoint. The XML structure is as follows:

```
<?xml version="1.0" encoding="UTF-8"?>
<application>
  <name>CUENTA</name>
  <instance>
    <instanceId>856d1d6dd9ec:cuenta:8080</instanceId>
    <hostName>172.19.0.6</hostName>
    <app>CUENTA</app>
    <ipAddr>172.19.0.6</ipAddr>
    <status>UP</status>
    <overriddenstatus>UNKNOWN</overriddenstatus>
    <port enabled="true">8080</port>
    <securePort enabled="false">443</securePort>
    <countryId>1</countryId>
    <dataCenterInfo class="com.netflix.appinfo.InstanceInfo$DefaultDataCenterInfo">
      <name>MyOwn</name>
    </dataCenterInfo>
    <leaseInfo>
      <renewalIntervalInSecs>5</renewalIntervalInSecs>
      <durationInSecs>15</durationInSecs>
      <registrationTimestamp>1791169435269</registrationTimestamp>
      <lastRenewalTimestamp>1791169510101</lastRenewalTimestamp>
      <evictionTimestamp>0</evictionTimestamp>
      <serviceUpTimestamp>1791169434928</serviceUpTimestamp>
    </leaseInfo>
    <metadata>
      <management.port>8080</management.port>
    </metadata>
    <homePageUrl>http://172.19.0.6:8080/</homePageUrl>
    <statusPageUrl>http://172.19.0.6:8080/actuator/info/</statusPageUrl>
    <healthCheckUrl>http://172.19.0.6:8080/actuator/health/</healthCheckUrl>
    <vipAddress>cuenta</vipAddress>
    <secureVipAddress>cuenta</secureVipAddress>
    <isCoordinatingDiscoveryServer>false</isCoordinatingDiscoveryServer>
    <lastUpdatedTimestamp>1791169435269</lastUpdatedTimestamp>
    <lastDirtyTimestamp>1791169434885</lastDirtyTimestamp>
    <actionType>ADDED</actionType>
  </instance>
</application>
```

Piensa. No, nadie usa todavía el directorio para encontrar a cuenta. Los servicios solo se anotan. Transferencia sigue llamando a `http://cuenta:8080`, y ese nombre lo resuelve el DNS de Docker, no Eureka.

Paso 3: Gateway

Hora: 9:07 p.m

Antes de poner el gateway llamé directo a movimiento con `curl -i -X POST http://localhost:8082/movimientos`. Respondió 400 y no 404, lo que significa que el endpoint existe y cualquiera lo puede llamar, aunque registrar movimientos debería ser una operación solo interna.

```
PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
● $ curl.exe -i -X POST http://localhost:8082/movimientos
HTTP/1.1 400
Content-Type: application/problem+json
Transfer-Encoding: chunked
Date: Mon, 05 Oct 2026 03:08:09 GMT
Connection: close

{"type":"about:blank","title":"Bad Request","status":400,"detail":"Failed to read request","instance":"/movimientos"}
```

1. En el `application.yml` del gateway agregué las rutas de movimiento, transferencia, notificación y SPEI debajo de la de cuenta. Movimiento y notificación solo aceptan GET, y SPEI solo acepta POST.
2. En `docker-compose.yml` descomenté el servicio gateway, que publica el puerto 8080.
3. Lo levanté con `docker compose up -d --build gateway` y en `localhost:8761` apareció un sexto renglón: GATEWAY.

Las pruebas por el gateway. Hice dos peticiones y las dos respondieron 404, pero por razones distintas:

- `GET /api/cuentas/000` sí trajo la cabecera `X-Instancia`, así que el 404 lo produjo cuenta. El gateway encontró la ruta, le preguntó al directorio y cuenta respondió que esa cuenta no existe.
- `POST /api/movimientos` no trajo `X-Instancia`, así que el 404 lo produjo el gateway. Ninguna ruta acepta POST a movimientos y la petición nunca llegó al servicio.

Comparado con el 400 del puerto 8082, la operación interna ya no se puede alcanzar desde afuera.

```
PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
● $ curl -i http://localhost:8080/api/cuentas/000
HTTP/1.1 404 Not Found
transfer-encoding: chunked
X-Instancia: 856d1d6dd9ec
Content-Type: application/problem+json
Date: Mon, 05 Oct 2026 03:14:32 GMT

{"type":"about:blank","title":"Not Found","status":404,"detail":"No existe la CLABE 000","instance":"/cuentas/000"}
```

```
PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
● $ curl -i -X POST http://localhost:8080/api/movimientos
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 129

{"timestamp":"2026-10-05T03:14:39.790+00:00","path":"/api/movimientos","status":404,"error":"Not Found","requestId":"f171e7f4-6"}
```

Mi cuenta. Creé mi propia cuenta por el gateway con la CLABE. El alta respondió 201 y la consulta respondió 200, con saldo 0.

```
PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
● $ curl -i -X POST http://localhost:8080/api/cuentas \
  > -H 'Content-Type: application/json' \
  > -d '{"clabe\\":\\"$LAB_CLABE\\",\\"titular\\":\\"S07\\",\\"saldoInicial\\":0}'
HTTP/1.1 201 Created
transfer-encoding: chunked
X-Instancia: 856d1d6dd9ec
Content-Type: application/json
Date: Mon, 05 Oct 2026 03:16:31 GMT

{"id":1,"clabe":"S071791170164","titular":"S07","saldo":0}
PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
● $ URL="http://localhost:8080/api/cuentas/$LAB_CLABE"

PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
● $ curl -i "$URL"
HTTP/1.1 200 OK
transfer-encoding: chunked
X-Instancia: 856d1d6dd9ec
Content-Type: application/json
Date: Mon, 05 Oct 2026 03:16:45 GMT

{"id":1,"clabe":"S071791170164","titular":"S07","saldo":0.00}
PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
● $ echo "$URL"
http://localhost:8080/api/cuentas/S071791170164
```

Pasos 4 y 5: Copias y caída

Paso 4: Tres copias de cuenta

Hora: 9:18 p.m

En docker-compose.yml cambié los ports de cuenta por deploy con replicas: 3. También borré los ports de movimiento, transferencia, notificación y SPEI, para que solo quedaran publicados el gateway en el 8080 y el directorio en el 8761. Apliqué el cambio con docker compose up -d.

Con docker ps --filter name=cuenta vi las tres copias, cada una con 8080/tcp sin publicar.

En localhost:8761, CUENTA pasó a UP con tres copias. Nadie le avisó al gateway que ahora había tres: lo supo porque las tres se anotaron en el directorio.

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
CUENTA	n/a (3)	(3)	UP (3) - ecda49c04ad2:cuenta:8080 , 8815c0b9ff99:cuenta:8080 , 06cad2d4b73d:cuenta:8080
GATEWAY	n/a (1)	(1)	UP (1) - 5cda41d28f7a:gateway:8080
MOVIMIENTO	n/a (1)	(1)	UP (1) - 54c552bcc100:movimiento:8080
NOTIFICACION	n/a (1)	(1)	UP (1) - 7ebde1652e55:notificacion:8080
SPEI	n/a (1)	(1)	UP (1) - 011bc8930ad8:spei:8080
TRANSFERENCIA	n/a (1)	(1)	UP (1) - 9918b2097345:transferencia:8080

Consulté mi cuenta nueve veces con un ciclo for y la cabecera X-Instancia se fue turnando entre las tres copias: [A, B, C, A, B, C, A, B, C]. Los valores coinciden con los ID de la tabla. Este reparto por turnos se llama round robin.

```
PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
$ for i in {1..9}; do
> curl -sS -D - -o /dev/null "$URL" | grep -i '^X-Instancia'
> done
X-Instancia: 06cad2d4b73d
X-Instancia: 8815c0b9ff99
X-Instancia: ecda49c04ad2
X-Instancia: 06cad2d4b73d
X-Instancia: 8815c0b9ff99
X-Instancia: ecda49c04ad2
X-Instancia: 06cad2d4b73d
X-Instancia: 8815c0b9ff99
X-Instancia: ecda49c04ad2
```

En otra terminal dejé corriendo docker compose logs -f cuenta | grep atendida y vi cómo cuenta-1, cuenta-2 y cuenta-3 atendían las peticiones una por una.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose logs -f cuenta | Select-String "atendida"
cuenta-2 | 2026-10-05T03:23:35.392Z INFO 1 --- [cuenta] [nio-8080-exec-9] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 8815c0b9ff99
cuenta-2 | 2026-10-05T03:23:35.690Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 8815c0b9ff99
cuenta-2 | 2026-10-05T03:23:35.871Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 8815c0b9ff99
cuenta-2 | 2026-10-05T03:23:54.114Z INFO 1 --- [cuenta] [nio-8080-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 8815c0b9ff99
cuenta-2 | 2026-10-05T03:23:54.300Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 8815c0b9ff99
cuenta-2 | 2026-10-05T03:23:54.440Z INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 8815c0b9ff99
cuenta-1 | 2026-10-05T03:23:35.242Z INFO 1 --- [cuenta] [nio-8080-exec-9] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 06cad2d4b73d
cuenta-1 | 2026-10-05T03:23:35.621Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 06cad2d4b73d
cuenta-1 | 2026-10-05T03:23:35.809Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 06cad2d4b73d
cuenta-1 | 2026-10-05T03:23:54.051Z INFO 1 --- [cuenta] [nio-8080-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 06cad2d4b73d
cuenta-1 | 2026-10-05T03:23:54.237Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 06cad2d4b73d
cuenta-1 | 2026-10-05T03:23:54.392Z INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por 06cad2d4b73d
cuenta-3 | 2026-10-05T03:23:35.111Z INFO 1 --- [cuenta] [nio-8080-exec-9] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por ecda49c04ad2
cuenta-3 | 2026-10-05T03:23:35.545Z INFO 1 --- [cuenta] [io-8080-exec-10] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por ecda49c04ad2
cuenta-3 | 2026-10-05T03:23:35.746Z INFO 1 --- [cuenta] [nio-8080-exec-1] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por ecda49c04ad2
cuenta-3 | 2026-10-05T03:23:54.175Z INFO 1 --- [cuenta] [nio-8080-exec-6] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por ecda49c04ad2
cuenta-3 | 2026-10-05T03:23:54.346Z INFO 1 --- [cuenta] [nio-8080-exec-7] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por ecda49c04ad2
cuenta-3 | 2026-10-05T03:23:54.486Z INFO 1 --- [cuenta] [nio-8080-exec-8] mx.mexibanco.cuenta.CabeceraInstancia : GET /cuentas/S071791170164 atendida por ecda49c04ad2
```

Paso 5: Matar una copia

Hora: 9:26 p.m

Mi predicción. Pensé que fallarían solo algunas consultas, más o menos una de cada tres, porque el gateway seguiría mandando peticiones a la copia muerta. Calculé que los errores durarían entre 15 y 20 segundos, hasta que el directorio la borrara.

En una terminal dejé corriendo una consulta por segundo durante 60 segundos. En otra maté una copia de golpe con docker kill mexi-banco-v06-cuenta-1.

Lo que medí.

- Apagué la copia a las **21:28:17**.
- CUENTA bajó de tres a dos copias en el tablero a las **9:29 p.m** es decir, tardó **1 minuto** en salir del directorio.

- Vi 0 errores 500 o 503 de 60 consultas.

```
PC@YUICHI MINGW64 ~/Desktop/MexoBanco-S07/mexi-banco (trabajo-s07)
$ for i in {1..60}; do
> echo "$(date +%T)  ${curl -s -o /dev/null -w '%{http_code}' --max-time 3 "$URL")"
> sleep 1
> done
21:28:25  200
21:28:26  200
21:28:27  200
21:28:28  200
21:28:29  200
21:28:30  200
21:28:31  200
21:28:33  200
21:28:34  200
21:28:35  200
21:28:36  200
21:28:37  200
21:28:38  200
```

Por qué pasó. Nadie le avisó al sistema que la copia murió; se fue enterando por capas. El directorio dejó de recibir heartbeats, pero esperó el plazo de 15 segundos y su revisión periódica de cada 5 segundos antes de borrarla.

El gateway no ve el directorio en vivo: guarda una copia de la lista, la renueva cada 5 segundos y además tiene 5 segundos de caché en el balanceador. Mientras tanto, por round robin, una de cada tres peticiones seguía yendo a la copia muerta y fallaba.

Acerté en que el sistema no se caería por completo: las otras dos copias siguieron atendiendo y mi cuenta respondió todo el tiempo. No acerté en los errores, porque esperaba que fallara una de cada tres consultas durante 15 a 20 segundos y vi 0 errores en 60 consultas. Tampoco acerté en el tiempo: la copia tardó alrededor de un minuto en salir del directorio, no de 15 a 20 segundos. No sé con certeza por qué no vi errores. Una posibilidad es que el gateway reintentara esas peticiones en otra copia viva. Para confirmarlo, repetiría la prueba anotando los segundos exactos. Al final reviví la copia con docker start mexi-banco-v06-cuenta-1 y CUENTA volvió a tener tres copias en UP.

Cierre

Hora: 9:33 p.m

- **El directorio resuelve** saber qué servicios existen y dónde está cada uno, sin escribir direcciones a mano. Cada servicio se anota solo y avisa con heartbeats que sigue vivo.
- **El gateway resuelve** que el cliente tenga una sola puerta, el puerto 8080, con reglas de qué se puede llamar desde afuera. Lo que no tiene ruta no pasa, como el POST a movimientos.
- **El balanceador resuelve** repartir las peticiones por turnos entre varias copias de un servicio, y dejar de mandarle a una copia cuando sale del directorio.
- **Cuando murió una copia vi 0 errores** intermitentes durante unos **15 a 20 segundos, porque** el directorio tardó en borrarla y el gateway seguía usando su lista guardada. Por turnos, una de cada tres peticiones iba a la copia muerta.

Tarea: Que transferencia también reparta

Hora: 9:36 p.m

Transferencia llamaba a http://cuenta:8080. Ese nombre lo resolvía el DNS de Docker, así que transferencia no sabía cuántas copias de cuenta había ni cuáles estaban vivas. Si una copia moría, transferencia no se enteraba.

El objetivo: Que transferencia llame solo por el nombre http://cuenta, que su propio balanceador decida a qué copia va usando la lista del directorio, y que una copia muerta salga de esa lista cuando el directorio la borre.

Lo que hice.

1. En ClientesHttp.java de transferencia agregué los imports de RestClientBuilderConfigurer y LoadBalanced. En el lugar del TODO H3 agregué un RestClient.Builder con la anotación @LoadBalanced, que en cada petición cambia el nombre cuenta por la dirección de una de sus copias registradas.
2. En docker-compose.yml dejé las URLs de transferencia solo con el nombre: http://cuenta, http://movimiento y http://notificacion.
3. Reconstruí con docker compose up -d --build y esperé a que todo estuviera en healthy.

Resultado. Corrí ./demo-v06.sh. Los saldos quedaron en 800/700 después de la transferencia y en 750/700 después del SPEI. El SPEI repetido con el mismo ID no cobró dos veces, y los errores 404 y 422 no movieron saldos.

```
./usr/bin/bash --login -i C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco\demo-v06.sh

-- 0. Esperar a que el directorio conozca a todos y la puerta encuentre a cuent
a --
-> lista en 1 s. lo que sabe el directorio (http://localhost:8761/eureka/apps):
CUENTA      3 instancia(s)
GATEWAY     1 instancia(s)
MOVIMIENTO  1 instancia(s)
NOTIFICACION 1 instancia(s)
SPEI        1 instancia(s)
TRANSFERENCIA 1 instancia(s)
-> 404 de cuenta (no soy del gateway) para una CLASE que no existe: la puerta ya sabe llegar

-- 1. Abrir la cuenta de Ana con 1000 (por la puerta: POST /api/cuentas) --
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009117205501","titular":"Ana","saldoInicial":1000}'
{"id":"22","clave":"002180009117205501","titular":"Ana","saldo":1000}
[HTTP 201]
-> el gateway quitó /api y lo mando a lb://cuenta: a una de sus tres replicas

-- 2. Abrir la cuenta de Beto con 500 (misma puerta) --
$ curl -X POST http://localhost:8080/api/cuentas -H 'Content-Type: application/json' -d '{"clave":"002180009117205502","titular":"Beto","saldoInicial":500}'
{"id":"23","clave":"002180009117205502","titular":"Beto","saldo":500}
[HTTP 201]

-- 3. Transferir 200 de Ana a Beto (misma puerta: POST /api/transferencias) --
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009117205501","claveDestino":"002180009117205502","monto":200}'
{"type":"about:blank","title":"Service unavailable","status":503,"detail":"No se pudo contactar a otro servicio: I/O error on GET request for \"/http://cuenta/cuentas/002180009117205501/": Connection refused","instance":"/transferencias"}
[HTTP 503]
-> transferencia llama a http://cuenta (un nombre, sin puerto). A que replicas fue cada llamada:
-> consulta de origen y destino, cargo y abono: cada llamada la reparte el balanceador de transferencia

-- 4. Consultar esa transferencia --
$ curl http://localhost:8080/api/transferencias/
{"type":"about:blank","title":"Not Found","status":404,"detail":"No static resource transferencias","instance":"/transferencias/"}
[HTTP 404]

-- 5. Saldos: el dinero no se crea ni se pierde --
$ curl http://localhost:8080/api/cuentas/002180009117205501
{"id":"22","clave":"002180009117205501","titular":"Ana","saldo":1000.00}
[HTTP 200]
$ curl http://localhost:8080/api/cuentas/002180009117205502
{"id":"23","clave":"002180009117205502","titular":"Beto","saldo":500.00}
[HTTP 200]
-> Ana 1000.00 + Beto 500.00 = 1500.00 (antes: 1000 + 500 = 1500). Esperado: 800 y 700

-- 6. Estado de cuenta de Ana y avisos de Beto (GET /api/movimientos, GET /api/notificaciones) --
$ curl http://localhost:8080/api/movimientos?clave=002180009117205501
[{"id":"28","claveCuenta":"002180009117205501","tipo":"DEPOSITO","monto":1000.00,"saldoResultante":1000.00,"fecha":"2026-10-05T03:47:36.243595Z","referencia":"saldo_inicial"}]
[HTTP 200]
$ curl http://localhost:8080/api/notificaciones?clave=002180009117205502
[]
[HTTP 200]

-- 7. SPEI de 50 con idempotency-key, dos veces (POST /api/spei) --
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-key: spei-demo-91172055' -d '{"claveOrigen":"002180009117205501","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{"id":"17","idempotencyKey":"spei-demo-91172055","claveOrigen":"002180009117205501","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50,"estado":"ENVIADO"}
[HTTP 201]
$ curl -X POST http://localhost:8080/api/spei -H 'Content-Type: application/json' -H 'Idempotency-key: spei-demo-91172055' -d '{"claveOrigen":"002180009117205501","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50}'
{"id":"17","idempotencyKey":"spei-demo-91172055","claveOrigen":"002180009117205501","bancoDestino":"BANCO-X","claveDestino":"0121800000000000009","monto":50.00,"estado":"ENVIADO"}
[HTTP 201]
-> mismo id (17 -> 17): no se cobro dos veces. Saldo de Ana: 950.00 (esperado 750)

-- 8. Round robin: 9 consultas seguidas a la misma cuenta, por la misma puerta --
consulta 1 X-Instancia: ecda49c04ad2 (mexi-banco-v06-cuenta-3)
consulta 2 X-Instancia: 8811c0b9f799 (mexi-banco-v06-cuenta-2)
consulta 3 X-Instancia: 06cad2d4b73d (mexi-banco-v06-cuenta-1)
consulta 4 X-Instancia: 8811c0b9f799 (mexi-banco-v06-cuenta-2)
consulta 5 X-Instancia: 06cad2d4b73d (mexi-banco-v06-cuenta-1)
consulta 6 X-Instancia: ecda49c04ad2 (mexi-banco-v06-cuenta-3)
consulta 7 X-Instancia: 8811c0b9f799 (mexi-banco-v06-cuenta-2)
consulta 8 X-Instancia: 06cad2d4b73d (mexi-banco-v06-cuenta-1)
consulta 9 X-Instancia: ecda49c04ad2 (mexi-banco-v06-cuenta-3)
-> 9 repeticiones distintas en 9 consultas: 0 veces se repitió la misma dos veces seguidas
-> el cliente no sabe cuantas replicas hay ni donde estan: solo conoce al 8080

-- 9. Los endpoints internos no tienen puerta --
$ curl -X POST http://localhost:8080/api/transferencias/002180009117205501/abonos -H 'Content-Type: application/json' -d '{"monto":1000000}'
{"timestamp":"2026-10-05T03:47:42.667+00:00","path":"/api/cuentas/002180009117205501/abonos","status":404,"error":"Not Found","requestID":"7140ba13-738b"}
[HTTP 404]
-> 404: el gateway no tiene ruta para /cargos ni /abonos. Adentro de la red siguen abiertos (seguridad: mas adelante)

-- 10. Errores de negocio, por la puerta, con el mismo código que dio cuenta --
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009117205501","claveDestino":"002180009117205599","monto":10}'
{"type":"about:blank","title":"Service unavailable","status":503,"detail":"No se pudo contactar a otro servicio: I/O error on GET request for \"/http://cuenta/cuentas/002180009117205501/": Connection refused","instance":"/transferencias"}
[HTTP 503]
$ curl -X POST http://localhost:8080/api/transferencias -H 'Content-Type: application/json' -d '{"claveOrigen":"002180009117205501","claveDestino":"002180009117205502","monto":99999}'
{"type":"about:blank","title":"Service unavailable","status":503,"detail":"No se pudo contactar a otro servicio: I/O error on GET request for \"/http://cuenta/cuentas/002180009117205501/": Connection refused","instance":"/transferencias"}
[HTTP 503]

-- 11. Los saldos no se movieron con los errores --
```

Con docker compose logs cuenta vi que el cargo y el abono de una misma transferencia los atendieron copias distintas.

```
PS C:\Users\PC\Desktop\MexoBanco-S07\mexi-banco> docker compose logs cuenta | select-string "cargos|abonos"
cuenta-3 | 2026-10-05T03:41:38.091Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009117169201/cargos atendida por ecda49c04ad2
cuenta-3 | 2026-10-05T03:41:50.775Z INFO 1 --- [cuenta] [nio-8080-exec-3] mx.mexibanco.cuenta.CabeceraInstancia : POST /cuentas/002180009117170601/cargos atendida por ecda49c04ad2
```